Introduction to Electronic Design Automation

Spring 2026
National Taiwan University

Problem Set 3

Due by 2025/06/05 23:59.

1 [BDD Operation]

[15%] Let $f = ab(\neg c + d) + (a\neg b + \neg ab)(c\neg d + \neg cd)$.

(a) (5%) Derive the ROBDD of f using procedure `robdd_build` with variable ordering $a < b < c < d$ (with a on top).
(b) (5%) Draw the **shared** ROBDDs of f, f_c and $f_{\neg c}$ under variable ordering $a < b < c < d$ (with a on top).
(c) (5%) Apply the ITE operation on the above ROBDDs to compute the ROBDD of $\exists c.f$.

2 [SAT Solving]

[20%] Consider SAT solving the CNF formula consisting of the following clauses. Assume the decision order follows a, b, c, d, and then e; assume each variable is assigned 0 first and then 1.

$$C_1 = (a + b + c), C_2 = (a + \neg c + \neg d + e), C_3 = (\neg b + c + \neg d + e),$$
$$C_4 = (a + \neg c + \neg e), C_5 = (\neg c + d + e), C_6 = (\neg b + c + d),$$
$$C_7 = (a + \neg b + c + \neg d + \neg e), C_8 = (\neg a + b + c + e), C_9 = (\neg a + \neg c + d + \neg e).$$

(a) (6%) Draw the search tree in solving the above CNF formula without implication and conflict-based learning.
(b) (6%) Draw the search tree in solving the above CNF formula with implication but without conflict-based learning.
(c) (8%) Draw the search tree in solving the above CNF formula with implication and conflict-based learning. Draw the implication graphs. Upon a conflict, show all possible learned clause candidates and choose one (with only one literal in the current decision level) to add to the clause set for subsequent solving.

3 [Combinational Equivalence Checking]

[25%] Consider the two circuits of Figure 1. Perform combinational equivalence checking on the two circuits with the following steps.

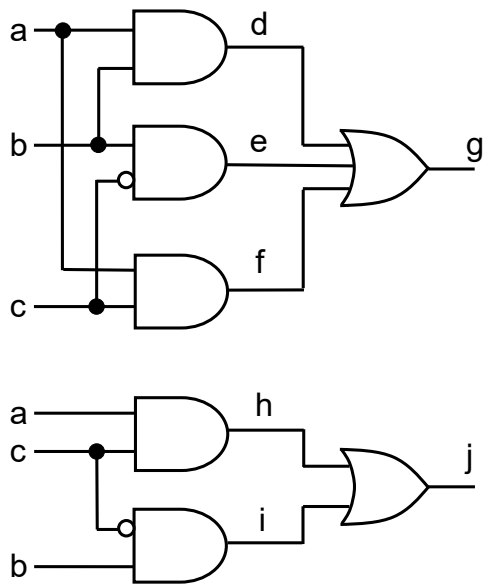

Fig. 1. Circuits under combinational equivalence checking.

(a) (3%) Construct the corresponding miter circuit for equivalence checking.
(b) (6%) Assume a computer word is of 8 bits. How many iterations of parallel (i.e., 8-bit) simulation are needed to achieve exhaustive simulation on the miter circuit constructed in (a)? Perform exhaustive parallel simulation on the miter circuit with the required number of iterations. For each iteration, derive the 8-bit signature (simulation patterns) for each gate (at the gate output) in the miter circuit. Determine from the signatures of the primary output of the miter circuit whether the two circuits under checking are equivalent.
(c) (6%) Let the output variable of the miter circuit constructed in (a) be z. Translate the equivalence checking problem into a Boolean satisfiability (SAT) instance in a CNF formula over the variables shown in Figure 1 and variable z.
(d) (5%) Write the CNF formula into a .cnf file in the DIMACS format[1] (Note that you may need to rename the signals as numbers to fit the DIMACS

[1] **DIMACS CNF format** is widely accepted as the standard format for Boolean formulas in CNF. In such a file, a line of comment starts with "c;" the number of variables and the number of clauses is defined by the line:

```
p cnf #variables #clauses
```

Each of the next lines specifies a clause: a positive literal is denoted by the corresponding number, and a negative literal is denoted by the corresponding negative number. The last number in a line should be zero. E.g.,

```
c A sample .cnf file.
p cnf 3 2
1 -3 0
2 3 -1 0
```

CNF format.) Run MiniSAT (`http://minisat.se/`), a popular SAT solver, on your .cnf file. Attach your .cnf file and the solving result.

(e) (5%) Write the two circuits of Figure 1 in the `blif` format (`https://www.cse.iitb.ac.in/~supratik/courses/cs226/spr16/blif.pdf`) and use the `cec` command in ABC (`https://people.eecs.berkeley.edu/~alanmi/abc/`) for equivalence checking. Attach your .blif files and the solving result.

4 [Characteristic Function]

[15%] Given a state transition relation $T(\boldsymbol{s}, \boldsymbol{s}')$ and a characteristic function $C(\boldsymbol{s})$ of some care states, where $\boldsymbol{s}$ and $\boldsymbol{s}'$ are vectors of current-state and next-state variables, respectively, derive the characteristic functions of the following state sets:

(a) (5%) The set of states each of which has no direct transition to itself (i.e., no self-transition).
(b) (5%) The set of states that can be reached from C in exactly two transitions.
(c) (5%) The set of states that each have a unique next state.

Express each of the above sets in a single quantified Boolean formula. (Hint: You may introduce new variables. Also, note that quantified variables can be arbitrarily renamed, e.g., $\exists x.f(x, y) = \exists z.f(z, y)$.)

5 [Sequential Equivalence Checking]

[25%] Given two sequential circuits C_1 and C_2 of Figure 2, perform sequential equivalence checking with the following steps.

(a) (4%) Derive the transition functions and output functions of C_1 and C_2. Assuming the initial values of the registers r_1, r_2, r_3, and r_4 are 0, 1, 0, and 0, respectively, derive the characteristic functions of the initial states of C_1 and C_2. (Let the current-state variables be s_1, s_2, s_3, s_4 and next-state variables be s'_1, s'_2, s'_3, s'_4.)
(b) (4%) Derive the transition and output functions of the product machine $C_{1\times 2}$ of C_1 and C_2; derive the initial state of $C_{1\times 2}$.
(c) (5%) Derive the Boolean expression for the transition relation $T(\boldsymbol{x}, \boldsymbol{s}, \boldsymbol{s}')$ of $C_{1\times 2}$ (with the input variables), and transition relation $T_\exists(\boldsymbol{s}, \boldsymbol{s}') = \exists \boldsymbol{x}.T(\boldsymbol{x}, \boldsymbol{s}, \boldsymbol{s}')$ (without the input variables).
(d) (6%) Perform reachability analysis using $T_\exists$ and compute the reached state set in every iteration in terms of the characteristic function. (There is no need to show the details of formula manipulation.) Conclude the equivalence between C_1 and C_2 using the information of the reachable states.
(e) (6%) Repeat (d) with the initial values 0, 1, 1, and 0, for registers r_1, r_2, r_3, and r_4, respectively.

More examples can be found in `https://www.cs.ubc.ca/~hoos/SATLIB/benchm.html`.

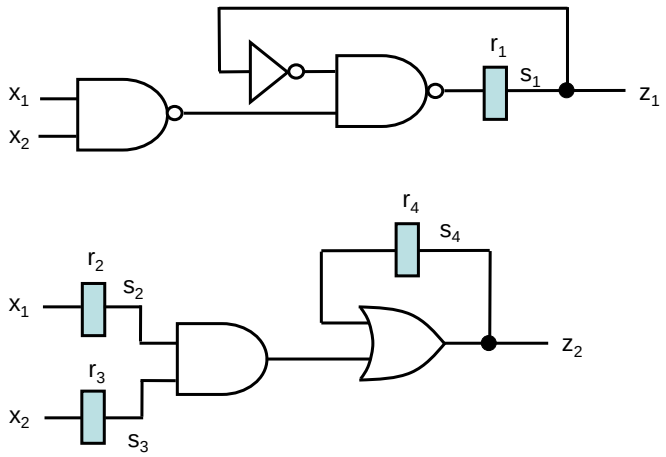

Fig. 2. Two circuits C_1 (upper) and C_2 (lower) under sequential equivalence checking.